Attention Is All You Need

I started this paper because it felt like the right place to begin. If I want to go deeper into applied ML systems and understand how modern LLMs, RAG systems, and agentic workflows actually work under the hood, then avoiding this paper would be impossible.

Most of the AI systems we talk about today, from GPT-style models to retrieval-based applications, are standing on top of the Transformer architecture. So this paper is not just a “famous ML paper.” It is one of those papers that changed the direction of the field.

My goal here is not to explain it like a researcher. I am reading it as an engineer who wants to understand why this architecture mattered, what problem it solved, and how the ideas connect to real systems.

The problem before Transformers

Before Transformers, sequence models like RNNs and LSTMs were commonly used for language tasks.

The main idea behind those models was simple. If you have a sentence, you process it step by step. One word goes in, the model updates its hidden state, then the next word goes in, and so on.

For example:

“The cat sat on the mat.”

An RNN reads “The,” then “cat,” then “sat,” and continues one token at a time. The model carries information forward through its hidden state.

This works, but it creates a major limitation. The model has to move through the sequence in order. It cannot easily look at the whole sentence at once.

That becomes a problem when relationships are far apart.

For example:

“The book that the professor recommended during the lecture was difficult.”

The word “book” and the word “was” are connected, but there are many words between them. In an RNN-like model, information has to travel through all those intermediate steps. By the time the model reaches the later word, some useful context may be weakened or lost.

LSTMs were designed to help with this problem, but they did not fully remove the sequential bottleneck. The model still processed tokens one after another.

That is the big shift this paper introduced.

Instead of processing a sequence step by step, the Transformer allows tokens to directly look at other tokens.

The Core Idea

The most important idea in the paper is attention.

In simple terms, attention means:

Each word looks at the other words and decides which ones matter most.

This mechanism is called self-attention, since each token attends to other tokens within the same sequence rather than relying on an external memory or sequential state.

That is the heart of the Transformer.

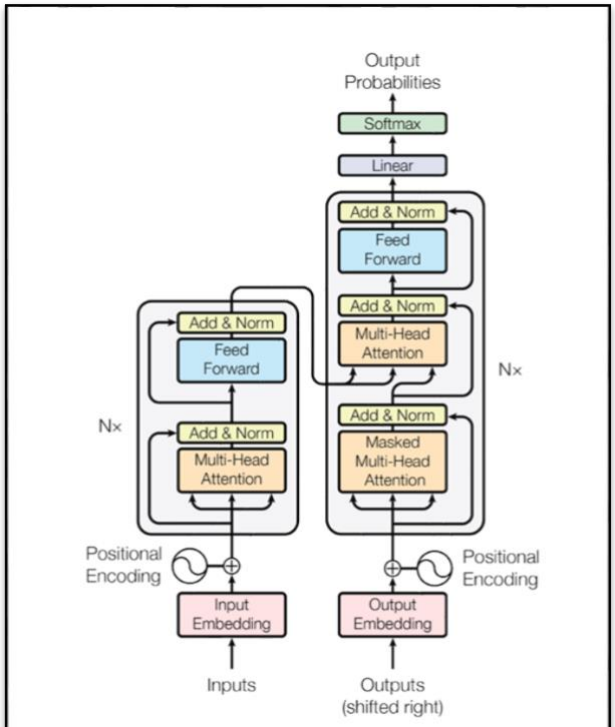

Figure 1: The Transformer - model architecture.

If the sentence is: “The cat sat on the mat.”

When the model is processing the word “sat,” it should probably pay more attention to “cat” than to “the.” The word “cat” tells us who is doing the action. The word “the” is useful grammatically, but less important for meaning.

Attention gives the model a way to assign different weights to different words.

This is powerful because the model does not need to wait for information to travel through a chain of hidden states. It can directly compare tokens with each other.

That is where the name of the paper comes from. The authors showed that you do not need recurrence or convolution to build a strong sequence model. Attention itself can be the main mechanism. Also, one thing that stood out to me is that since Transformers process tokens in parallel, they don't naturally understand order. Unlike RNNs, there is no built-in notion of sequence. To fix this, the model adds positional encoding to each token embedding so it knows where each word appears in the sentence.

Query, Key, and Value

The paper introduces attention through three vectors:

- Query
- Key
- Value

At first these names sound abstract, but the idea is not too complicated.

- A query means: "What am I looking for?"
- A key means: "What kind of information do I represent?"
- A value means: "What information do I actually carry?"

Imagine you are in a room asking a question. Your question is the **query**.

Each person in the room has a name tag describing what they know. That is the **key.**

Each person also has the actual information they can give you. That is the **value**.

1. You compare your query with everyone's key.
2. The people whose key matches your query more closely get more attention.
3. Then you use their values more heavily to form your answer.

That **is roughly what happens inside attention**.

For each token, the model creates query, key, and value vectors. Then it compares queries with keys to calculate attention scores. Those scores decide how much of each value should be used.

The attention equation

The main equation in the paper is:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This looks intimidating at first, but each part has a clear meaning.

$$QK^T$$

This compares queries with keys. It gives us similarity scores. The dot product here measures how aligned two vectors are. If the query and key point in similar directions, the score is higher, which means the model considers them more related. In higher dimensions, these dot products can become large, which is why the paper scales them down before applying softmax.

If a query and key point in a similar direction, their dot product is higher. That means the model thinks they are more related.

Then we divide by: $\sqrt{d_k}$

The division by √dk is important because as vector dimensions increase, dot product values can grow large, pushing softmax into extreme regions where it becomes less sensitive. Scaling keeps the distribution more stable and easier to train.

Then comes softmax: softmax(…)

Softmax converts raw scores into weights. These weights are between 0 and 1, and they add up to 1.

So instead of saying:

"cat = 12.4, mat = 3.1, the = 0.6"

the model turns it into something more like:

"cat = 0.75, mat = 0.20, the = 0.05"

This makes the scores usable as attention weights.

Finally, the model multiplies these weights by the values: ...V

That means the model combines information from different tokens based on how much attention each one received.

In simple words:

The model checks what matters, assigns importance, and then mixes information accordingly.

Why multi-head attention matters

The paper does not use just one attention operation. It uses multi-head attention. This is one of the most interesting parts.

One attention head may learn one type of relationship. Another head may learn something else. Each head projects the input into a different representation space before computing attention, which allows the model to capture different types of relationships in parallel.

For example, one head might focus on grammar. Another might focus on subject and verb relationships. Another might focus on positional patterns or meaning.

This is useful because language has many layers happening at the same time.

Take this sentence: “The doctor reviewed the report because it contained abnormal results.”

The word “it” refers to “report.” One attention head might learn that relationship. Another head might focus on the connection between “doctor” and “reviewed.” Another might focus on “abnormal results.”

A single attention head may not capture all of this well. Multi-head attention gives the model multiple ways to look at the same sentence.

As an engineer, I think this is one of the most elegant ideas in the paper. Instead of forcing one representation to capture everything, the model learns multiple perspectives in parallel.

Encoder and decoder

The Transformer architecture has an encoder and a decoder.

- The encoder reads the input and builds a contextual representation.
- The decoder generates the output.

For translation, this makes sense. If we are translating English to French, the encoder understands the English sentence, and the decoder generates the French sentence.

The decoder also uses masked attention. This means that when generating a word, the model cannot look at future words.

For example, if the output is being generated word by word, the model should not cheat by looking ahead at words it has not generated yet. Masked attention prevents that.

This matters because generation should happen in a realistic order. The model predicts the next token based only on what came before.

Another detail I found important is the use of residual connections and layer normalization throughout the architecture. These help stabilize training as the network gets deeper and make it easier for gradients to flow during backpropagation.

The real breakthrough

The paper is famous because of attention, but the deeper breakthrough is parallelization. RNNs process tokens one after another. Transformers process tokens in parallel. This made training much faster and more scalable.

That is the part that connects directly to modern AI. Large language models are possible because this architecture scales well. If models still had to process every token sequentially in the same way RNNs did, training massive models would be much harder.

So the paper did not just introduce a better NLP model. It changed what was computationally practical.

This is why I think the title is strong. “Attention Is All You Need” sounds almost too bold, but the paper proved that attention could become the core building block for sequence modeling.

What the paper helped me understand

Before reading this, I understood Transformers mostly at a high level. I knew they used attention, and I knew modern LLMs were based on them.

But reading the paper made the shift clearer. The model is not just “looking at words.” It is building relationships between tokens through learned similarity.

Every token is asking: “What should I pay attention to?”

That simple idea becomes powerful when scaled. It also made me understand why embeddings alone are not enough. A token embedding gives a representation of a token, but attention changes that representation based on context.

The word “bank” means something different in:

“I deposited money at the bank.”

and

“I sat near the river bank.”

Attention helps the model understand which surrounding words matter, so the representation becomes contextual.

That contextual understanding is one of the reasons Transformers became so important.

Where attention struggles

This paper is foundational, but attention is not perfect.

The biggest issue is that self-attention has quadratic complexity.

If the sequence length is n, attention compares tokens with other tokens, which leads to:

$$O(n^2)$$

That means if the input gets longer, the computation grows very quickly.

For short sentences, this is fine. For very long documents, codebases, medical records, or engineering histories, this becomes expensive.

This is one reason why long-context modeling is still a major area of research.

The second limitation is structure.

Transformers are very good at learning relationships from sequences, but they do not explicitly know structured relationships unless we provide them.

For example, in an engineering system, we may have:

- Service A depends on Service B
- Pull Request X modified Service A
- Incident Y happened after Pull Request X
- Decision Z explains why Service B was designed this way

These are not just pieces of text. These are relationships.

A Transformer can read text about these relationships, but it does not automatically maintain a persistent structured memory of them.

That is the gap I keep thinking about while building Engram.

Connection to Engram

Engram is about structured engineering memory. The idea is to take fragmented artifacts like pull requests, incidents, architectural decisions, and documentation, then model them as entities and relationships.

Reading this paper helps me understand what Transformers are great at and where they need support. Transformers are great at contextual understanding. But for system-level reasoning, I do not want the model to rely only on raw text.

If I ask: “What could break if this service changes?”

I do not want the model to guess from chunks of text. I want it to reason over explicit relationships:

- This service depends on that service
- This incident involved this component
- This decision affected this architecture
- This pull request changed this module

That is why Engram combines structured knowledge graphs with retrieval and LLM reasoning. The LLM should interpret and explain. It should not be the only place where memory exists. This paper makes that design choice feel even more important.

Attention is powerful, but structure still matters.

My takeaway

The biggest takeaway for me is that Transformers changed machine learning because they made context scalable. They allowed models to move away from step-by-step sequence processing and toward global attention over tokens. That shift created the foundation for modern LLMs.

But reading this paper also made me more aware of the next layer of the problem. Attention helps models understand relationships inside sequences. But real-world systems often need persistent, explicit relationships across time, tools, and artifacts.

That is where retrieval, graphs, memory layers, and agentic systems come in.

So for me, this paper is not just a historical milestone. It is the starting point for understanding why modern AI works, and also why modern AI still needs better grounding when applied to real systems.